Modul VIII
ListView dan GridView

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan perbedaan mekanisme pemuatan memori antara Eager Loading (ListView) dan Lazy Loading (ListView.builder).
2. Menerapkan ListView.builder dan ListView.separated untuk merender daftar data dinamis secara efisien.
3. Mengonfigurasi antarmuka kisi menggunakan GridView.builder dengan penentuan childAspectRatio yang tepat.
4. Menerapkan komponen Card dan ListTile untuk menyusun baris data yang rapi dan ergonomis.
5. Mendiagnosis dan memperbaiki error layout unbounded height menggunakan widget Expanded.
6. Mengintegrasikan class model data OOP Dart ke dalam antarmuka aplikasi secara modular.

B. Alat dan Bahan

Untuk melaksanakan praktikum menggunakan *framework* Flutter, diperlukan perangkat dan perangkat lunak pendukung sebagai berikut:

Perangkat Keras (Hardware)

Laptop/PC

- Prosesor: Minimal Intel Core i3 generasi ke-6 atau AMD Ryzen 3 setara (disarankan Intel Core i5/i7 atau Ryzen 5/7).
- RAM: Minimal 4 GB (disarankan 8 GB atau lebih untuk performa emulator yang lebih lancar).
- Penyimpanan: Minimal 10 GB ruang kosong (disarankan SSD 256 GB atau lebih).
- Kartu Grafis: Integrated GPU (Intel UHD/AMD Vega) sudah cukup, namun disarankan menggunakan dedicated GPU untuk mempercepat emulator.
- Layar: Resolusi minimal 1366 × 768 piksel (disarankan Full HD 1920 × 1080 piksel).
- Port USB/Type-C: Untuk menghubungkan perangkat Android fisik (opsional).

Perangkat Tambahan

- Smartphone Android/iOS (opsional, untuk pengujian langsung di perangkat nyata).
- Kabel data USB (opsional, jika menggunakan perangkat fisik).

Koneksi Internet Dibutuhkan untuk:

- Mengunduh Flutter SDK dan Android SDK.
- Mengunduh dependencies (package) saat pertama kali membuat proyek
- Flutter.
- Mengakses dokumentasi resmi Flutter.

Perangkat Lunak (Software)

Sistem Operasi

- Windows: Windows 10/11 (64-bit).
- Linux: Ubuntu 20.04 LTS atau distro lain (64-bit).
- macOS: Versi Catalina (10.15) atau lebih baru.

Flutter SDK (termasuk Dart SDK)

- Unduh dari: https://docs.flutter.dev/install
- Ekstrak dan tambahkan ke PATH sistem operasi.

Android SDK dan Android Studio

- Android Studio (opsional, digunakan untuk emulator Android).
- Jika tidak menggunakan emulator, dapat memakai perangkat Android fisik.

Editor Kode

- Visual Studio Code (VS Code) dengan ekstensi Flutter dan Dart
- Alternatif: Android Studio dengan *plugin* Flutter & Dart.

Emulator/Simulator (opsional, bila tidak ada perangkat fisik)

- Android Emulator (via Android Studio).
- iOS Simulator (hanya untuk macOS dengan Xcode).

C. Dasar Teori

C.1 Keterbatasan Widget Column dan Batasan Fisik Layar Smartphone

Pada materi Pekan 7, mahasiswa telah mempelajari teknik penyusunan elemen antarmuka secara linear menurun dari atas ke bawah menggunakan widget Column. Widget Column sangat andal digunakan untuk menyusun tata letak formulir masukan data yang pendek, kartu profil pengguna statis, atau kelompok tombol navigasi berukuran ringkas. Namun demikian, widget Column memiliki karakteristik arsitektur bawaan yang sangat kaku: bersifat statis dan sama sekali tidak memiliki kemampuan pengguliran layar (non-scrollable).

Layar fisik pada smartphone modern rata-rata hanya memiliki resolusi tinggi vertikal berkisar antara 700 hingga 900 density-independent pixels (dp). Ketika Anda menggunakan widget Column untuk menyusun puluhan elemen teks, kartu informasi, atau deretan gambar yang tinggi kumulatifnya melebihi batas fisik layar tersebut (sebagai contoh, total akumulasi tinggi konten mencapai 1.600 dp), Flutter Engine akan mengalami kebuntuan komputasi saat memetakan koordinat piksel pada lapisan rendering. Flutter Engine tidak diizinkan untuk secara otomatis memotong konten tanpa instruksi eksplisit dari programmer, dan di sisi lain layar fisik tidak memiliki ruang lagi untuk menggambar elemen tersebut. Akibatnya, Flutter Engine akan langsung melempar error visual fatal ke layar pengguna berupa garis strip kuning-hitam menyilang (hazard strip) yang disertai dengan pesan error pada konsol debug:

```
A RenderFlex overflowed by XXX pixels on the bottom.
The relevant error-causing widget was: Column
```

Banyak pengembang pemula mencoba mengakali masalah ini secara instan dengan membungkus widget Column tersebut ke dalam sebuah SingleChildScrollView. Pendekatan

ini memang dapat menghilangkan garis strip kuning-hitam untuk kasus data pendek (misalnya halaman pendaftaran yang panjangnya 2 atau 3 kali tinggi layar). Namun, membungkus Column dengan SingleChildScrollView untuk menampilkan data koleksi yang berjumlah puluhan, ratusan, hingga ribuan adalah sebuah kesalahan arsitektur fatal (anti-pattern). Hal ini terjadi karena Column tetap memaksakan seluruh anaknya dibuat secara bersamaan di memori RAM, yang pada akhirnya akan menghancurkan performa aplikasi pada smartphone pengguna. Untuk menampilkan sekumpulan data yang panjang, dinamis, dan tidak dapat diprediksi jumlahnya sejak awal, Flutter Framework menyediakan rumpun komponen penggulir khusus (scrollable widgets) dengan fondasi utamanya adalah keluarga widget ListView.

C.2 Arsitektur Internal Flutter Engine: Virtualization, Viewport, dan Cache Extent

Di balik layar, proses rendering pengguliran di dalam Flutter Framework diatur oleh tiga subsistem utama: Scrollable, Viewport, dan Sliver.

1. Scrollable: Subsistem yang bertugas mendeteksi dan menginterpretasikan gestur fisik jari pengguna di atas layar kaca (apakah pengguna melakukan slow drag, sentuhan cepat fling, atau menggeser melewati batas ujung layar overscroll).
2. Viewport: Jendela pandang visual virtual yang bertindak sebagai bingkai pembatas. Viewport hanya menampilkan irisan kecil dari total bentangan dimensi konten yang sebenarnya sangat panjang.
3. Sliver: Bagian-bagian kecil dari area pengguliran yang memiliki kemampuan untuk merender dirinya sendiri secara cerdas berdasarkan posisinya terhadap Viewport.

Berdasarkan cara mengalokasikan objek widget ke dalam memori RAM, mekanisme rendering daftar di Flutter dibagi secara tegas menjadi dua paradigma arsitektur:

C.3 Mekanisme Eager Loading (Pemuatan Seketika)

Ketika Anda menggunakan konstruktor default ListView(children: [...]), Anda mengirimkan sebuah array statis berisi objek-objek widget yang sudah jadi. Flutter Engine dipaksa untuk menginstansiasi seluruh objek widget tersebut ke dalam memori RAM sejak milidetik pertama halaman aplikasi dibuka.

Bayangkan sebuah aplikasi kependudukan Kota Bandung yang harus menampilkan riwayat 1.000 data warga. Jika Anda menggunakan ListView biasa, ke-1.000 widget kartu akan langsung dicetak di dalam memori RAM, dipetakan elemen pohonnya (Element Tree), dan dibuatkan objek visualnya (RenderObject), padahal mata pengguna di layar hanya sanggup membaca 6 sampai 8 baris data saja pada satu waktu. Akibatnya sangat fatal:

- Konsumsi Memori Melonjak Ekstrem: Alokasi memori RAM membengkak linier sebanding dengan jumlah data.
- Terjadinya Animasi Macet (Frame Drop / UI Jank): CPU dan GPU ponsel bekerja teramat keras pada milidetik pertama pembukaan halaman sehingga frame rate anjlok jauh di bawah standar 60 FPS.
- Pemborosan Daya Baterai: Ponsel cepat memanas dan menguras daya baterai pengguna.

C.4 Mekanisme Lazy Loading / On-Demand Rendering (Virtualisasi)

Konstruktor pintar ListView.builder menyelesaikan masalah di atas dengan menerapkan teknik virtualisasi antarmuka (interface virtualization). Pada mekanisme ini, Anda tidak memberikan daftar widget yang sudah jadi, melainkan memberikan sebuah fungsi callback generator yang bertindak layaknya pabrik perakitan.

Flutter Engine bertindak sangat efisien: widget hanya akan dirakit ketika baris indeks tertentu benar-benar hendak memasuki area tampilan layar (viewport).

Untuk menjamin pengguliran tetap terasa mulus tanpa jeda saat pengguna menggeser layar dengan cepat, Flutter Engine menerapkan konsep Penyangga Kejut (Cache Extent):

- Cache Extent Bawaan (250 piksel): Flutter secara otomatis menyiapkan area buffer kecil berukuran 250 piksel tepat di atas dan di bawah batas tepi Viewport. Widget yang berada pada zona 250 piksel ini sudah dirakit lebih awal sebelum mata pengguna melihatnya, sehingga transisi pengguliran berjalan sangat halus (smooth 60 FPS).
- Daur Ulang Elemen (Garbage Collection): Ketika jari pengguna menggulir layar ke atas, baris data di bagian atas akan terdorong menjauh melewati batas zona buffer 250 piksel. Pada detik itulah, Flutter Engine akan menghancurkan (deallocate) objek widget tersebut dan melepaskan memorinya agar dibersihkan oleh Dart Garbage Collector.

Dengan mekanisme virtualisasi ini, aplikasi yang menampilkan 10 data dan aplikasi yang menampilkan 100.000 data warga Kota Bandung akan mengonsumsi kapasitas memori RAM yang sama persis dan konstan.

C.5 Taksonomi Varian Konstruktor ListView dan Bedah Kode

Flutter Framework menyediakan tiga varian utama pada kelas ListView yang disesuaikan dengan kebutuhan kasus nyata:

1. Konstruktor Default: ListView(children: [...])

Varian ini menerima daftar widget statis secara langsung melalui properti children. Varian ini menerapkan Eager Loading murni.

- **Karakteristik:** Sangat mudah dipahami karena sintaksnya menyerupai widget Column. Cukup ganti kata Column menjadi ListView, maka seluruh elemen di dalamnya langsung memiliki kemampuan scroll vertikal.
- **Skenario Penggunaan:** Hanya gunakan konstruktor ini jika jumlah elemen sudah pasti sedikit (kurang dari 10 hingga 15 elemen), misalnya halaman pengaturan aplikasi (*Settings*), formulir pengubahan profil akun, atau menu bantuan.

```
ListView(
  padding: EdgeInsets.all(16),
  children: [
    Card(child: ListTile(title: Text('Pengaturan Notifikasi'))),
    Card(child: ListTile(title: Text('Privasi & Keamanan'))),
    Card(child: ListTile(title: Text('Bantuan & Dukungan Layanan'))),
    Card(child: ListTile(title: Text('Tentang Aplikasi Kota Bandung'))),
  ],
)
```

Penjelasan Program:

1. **ListView(...)**: Menginisialisasi kontainer penggulir linear satu dimensi yang membungkus seluruh anak di dalamnya.
2. **padding: EdgeInsets.all(16)**: Memberikan bantalan ruang kosong sebesar 16 piksel di sekeliling batas tepi dalam Viewport, mencegah kartu menempel kaku ke tepi kaca smartphone.
3. **children: [...]**: Menerima kumpulan widget kartu secara langsung. Keempat widget kartu ini langsung diinstansiasi ke dalam memori RAM secara serentak sejak halaman dimuat.

2. Konstruktor Pabrik: ListView.builder

Varian ini menerapkan Lazy Loading murni menggunakan fungsi generator.

- **Karakteristik:** Wajib memiliki parameter itemCount untuk membatasi jumlah baris dan fungsi itemBuilder untuk merakit widget per baris indeks.
- **Skenario Penggunaan:** Wajib digunakan setiap kali Anda berhadapan dengan data koleksi List yang panjang, data dinamis hasil pembacaan database SQLite/Hive, atau data yang diambil dari internet via REST API.

```
final List<String> kecamatanBandung = [
  'Coblong', 'Sukajadi', 'Cicendo', 'Sumur Bandung', 'Lengkong',
  'Cibeunying Kaler', 'Antapani', 'Batununggal', 'Regol', 'Buahbatu'
];

ListView.builder(
  padding: EdgeInsets.symmetric(horizontal: 16, vertical: 12),
  itemCount: kecamatanBandung.length,
  itemBuilder: (BuildContext context, int index) {
    return Card(
      margin: EdgeInsets.symmetric(vertical: 6),
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: Colors.teal.shade100,
          child: Text(
            '${index + 1}',
            style: TextStyle(color: Colors.teal.shade900, fontWeight:
FontWeight.bold),
          ),
        ),
        title: Text(
          kecamatanBandung[index],
          style: TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Text('Wilayah Administrasi Kota Bandung'),
        trailing: Icon(Icons.arrow_forward_ios, size: 14, color:
Colors.grey),
      ),
    );
  },
)
```

Penjelasan Program:

1. **final List<String> kecamatanBandung = [...]**: Mendeklarasikan sumber koleksi data mentah. Dalam lingkungan produksi nyata, list ini dapat menampung ribuan objek data yang bersumber dari server database.
2. **ListView.builder(...)**: Menginstansiasi komponen penggulir berbasis virtualisasi.
3. **itemCount: kecamatanBandung.length**: Menetapkan batas atas komputasi penggulir. Nilai ini sangat krusial bagi Flutter Engine untuk menghitung proporsi panjang bilah scrollbar (*scroll extent*). Jika parameter ini diabaikan (bernilai null), Flutter akan mengasumsikan daftar bersifat tak terhingga (*infinite scroll*) dan akan memanggil builder tanpa batas.
4. **itemBuilder: (BuildContext context, int index) { ... }**: Fungsi callback yang dieksekusi secara berulang oleh Flutter Engine hanya untuk baris-baris yang sedang atau akan terlihat di layar. Parameter index bertindak sebagai penunjuk iterasi otomatis yang berjalan mulai dari angka 00 hingga itemCount−1itemCount−1.
5. **kecamatanBandung[index]**: Mengambil elemen data string pada posisi array tertentu yang berkorespondensi langsung dengan baris yang sedang dirakit.
6. **leading: CircleAvatar(...)**: Merakit nomor urut baris di sisi kiri kartu dengan melakukan konversi visual ${index + 1} agar penomoran manusiawi dimulai dari angka 1, bukan dari angka 0.

3. Konstruktor Berpemisah: ListView.separated

Varian ini merupakan evolusi khusus dari ListView.builder yang dirancang untuk kasus di mana setiap baris data membutuhkan garis pembatas atau widget pemisah tertentu.

- **Karakteristik:** Memiliki parameter wajib tambahan bernama **separatorBuilder**.
- **Keunggulan Ergonomi Kode:** Jika menggunakan ListView.builder biasa, pengembang harus menyisipkan logika percabangan kondisional if yang rumit di dalam itemBuilder untuk mencegah garis pembatas muncul pada item paling terakhir. ListView.separated secara otomatis meniadakan garis pembatas di bawah item terakhir tanpa perlu logika manual!

```
ListView.separated(
  padding: EdgeInsets.all(16),
  itemCount: kecamatanBandung.length,
  separatorBuilder: (BuildContext context, int index) {
    if (index == 4) {
      return Container(
        margin: EdgeInsets.symmetric(vertical: 8),
        padding: EdgeInsets.all(8),
        color: Colors.amber.shade100,
        child: Text(
          '--- Batas Zona Wilayah Bandung Timur ---',
          textAlign: TextAlign.center,
          style: TextStyle(fontSize: 12, fontWeight: FontWeight.bold),
        ),
      );
    }
    return Divider(height: 1, thickness: 1, color: Colors.grey.shade300);
```

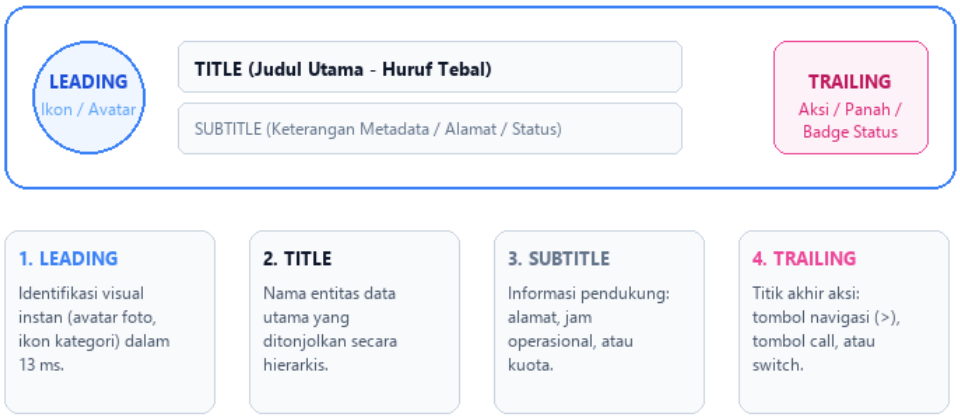
```
    },
    itemBuilder: (BuildContext context, int index) {
      return ListTile(
        title: Text(kecamatanBandung[index]),
        trailing: Icon(Icons.chevron_right),
      );
    },
  )
```

Penjelasan Program:

1. **ListView.separated(...)**: Mengaktifkan konstruktor penggulir dengan dua mesin perakit independen: perakit konten utama dan perakit garis pemisah.
2. **separatorBuilder: (BuildContext context, int index) { ... }**: Fungsi callback khusus yang hanya dipanggil di antara dua baris data. Nilai index pada separatorBuilder merepresentasikan indeks item data yang berada tepat di atas pemisah tersebut.
3. **if (index == 4) { return Container(...); }**: Mendemonstrasikan bahwa widget pemisah tidak terbatas pada garis tipis Divider, melainkan dapat diisi oleh widget kustom apa pun, seperti spanduk pengumuman wilayah atau penanda grup data.
4. **return Divider(...)**: Mengembalikan garis pembatas standar Material Design dengan ketebalan 1 piksel untuk indeks-indeks lainnya.

C.6 ListTile

Dalam disiplin interaksi manusia dan komputer (Human-Computer Interaction / HCI), perancangan antarmuka mobile berpedoman kuat pada Hukum Fitts (Fitts's Law): waktu yang dibutuhkan oleh jari manusia untuk membidik dan menekan target berbanding lurus dengan jarak target dan berbanding terbalik dengan ukuran fisik target tersebut. Ujung jari manusia rata-rata memiliki diameter sentuhan sekitar 10 hingga 14 mm. Pada layar smartphone, ukuran fisik ini setara dengan area sentuh minimal 48 × 48 density-independent pixels (dp). Menyusun baris daftar secara manual menggunakan kombinasi Row dan Column biasa rentan menghasilkan target sentuh yang terlalu tipis (misalnya baris teks setinggi 24 dp), sehingga memicu masalah salah pencet (fat-finger problem). Material Design menciptakan komponen baku bernama ListTile untuk memastikan setiap baris data mematuhi standar ergonomi mobile secara otomatis:



1. **leading (Zona Navigasi Visual Kiri):** Berdasarkan penelitian pelacakan mata pengguna (*eye tracking*), mata manusia memindai layar ponsel dengan pola huruf F (*F-Pattern*), dimulai dari sudut kiri atas. Slot leading menangkap fokus mata dalam 13 milidetik pertama untuk mengenali kategori data melalui bentuk visual, seperti CircleAvatar foto dokter, lambang ambulans, atau inisial nomor antrean.
2. **title (Zona Semantik Primer):** Slot untuk teks informasi terpenting dari entitas data, seperti nama rumah sakit atau nama layanan publik. Diberi gaya huruf tegas (*bold*) dan kontras tinggi.
3. **subtitle (Zona Informasi Sekunder):** Slot teks pendukung yang terletak persis di bawah title. Sangat ideal untuk menampilkan alamat jalan, jam buka operasional, atau lencana indikator status. Jika subtitle memiliki teks panjang yang memakan lebih dari 2 baris, Anda wajib menambahkan properti isThreeLine: true agar perataan vertikal komponen tetap seimbang.
4. **trailing (Zona Aksi Kanan):** Terletak di batas kanan baris data, tepat di bawah jangkauan ibu jari tangan kanan pengguna mobile. Digunakan untuk indikator navigasi rute halaman (Icon(Icons.arrow_forward_ios)) atau tombol aksi cepat (IconButton(Icons.phone)).
5. **onTap (Efek Sentuhan Animasi Material):** Ketika properti onTap: () { ... } didefinisikan, ListTile secara otomatis mengaktifkan lapisan InkWell internal yang memunculkan efek cipratan tinta (*ink splash ripple*) saat jari pengguna menyentuh baris tersebut, memberikan konfirmasi visual instan bahwa perintah sentuhan telah diterima oleh sistem.

C.7 GridView dan Peran gridDelegate

Ketika antarmuka aplikasi ditujukan untuk eksplorasi visual bebas (seperti katalog etalase produk UMKM, menu ikon dasbor layanan publik cerdas, atau galeri foto destinasi wisata Kota Bandung), penyajian linear vertikal satu dimensi pada ListView terasa lambat dan membosankan. Pengguna membutuhkan komponen kisi dua dimensi, yaitu **GridView**.

- **Perbedaan Psikologis Persepsi Visual Pengguna:**
- **ListView (Pola *Linear Scanning*):** Aliran mata bergerak satu arah dari atas ke bawah secara teliti untuk membaca informasi tekstual (misalnya riwayat mutasi bank, direktori nomor telepon darurat, atau log tiket laporan).
- **GridView (Pola *Visual Discovery*):** Aliran mata melompat secara bebas melintasi kisi-kisi kotak dua arah untuk mengenali objek grafis, warna kartu, dan bentuk ikon secara cepat.
- **Prinsip Pemisahan Tanggung Jawab (*Separation of Concerns*):**

Pada implementasi GridView.builder, Flutter menerapkan prinsip rekayasa perangkat lunak yang sangat elegan dengan memisahkan arsitektur ke dalam dua peran independen:

1. **Pabrik Konten (itemBuilder)**: Bertanggung jawab penuh atas **isi konten** visual kartu (warna latar, gambar ikon apa yang dipasang, judul apa yang dicetak, dan tombol apa yang disediakan).
2. **Arsitek Geometri (gridDelegate)**: Bertanggung jawab penuh atas **tata letak dan ruang** (berapa banyak kolom yang harus dibentuk, berapa lebar setiap kotak, berapa jarak celah spasi antar-kotak, dan pada koordinat X,YX,Y mana kotak tersebut harus digambar di layar).

Kelas delegasi layout yang paling lazim digunakan dalam industri Flutter adalah **SliverGridDelegateWithFixedCrossAxisCount**:

- **crossAxisCount**: Menentukan jumlah pembagian kolom tetap pada sumbu silang. Misalnya, nilai crossAxisCount: 2 berarti lebar layar smartphone akan dibagi rata menjadi 2 kolom vertikal.
- **crossAxisSpacing**: Ruang kosong horizontal yang memisahkan antara kolom sisi kiri dan kolom sisi kanan.
- **mainAxisSpacing**: Ruang kosong vertikal yang memisahkan antara baris atas dan baris bawah.

D. Tugas Praktikum

D.1 Panduan Penyiapan Proyek

Sebelum memulai pengerjaan Praktikum, mahasiswa wajib mengikuti tahapan penyiapan project flutter berikut secara berurutan:

1. Pengunduhan & Penataan Folder Proyek

- Unduh berkas arsip template praktikum berikut ini: https://drive.google.com/file/d/1a3Ej5UDewA1phmg_eoTxUuMxSK-3B3ty/view?usp=sharing
- Ekstrak berkas zip tersebut kemudian ganti dengan nama:
  - praktikum_08_nim
  - ta_08_nim
- Buka folder proyek tersebut menggunakan editor kode Visual Studio Code atau Android Studio.

2. Uji Coba Penjalaran Pertama Kali

Pastikan berkas skeleton dasar aplikasi dapat berjalan normal dan tampil di layar perangkat sebelum Anda melakukan modifikasi kode. Buka terminal di editor Anda, lalu jalankan perintah sesuai target platform yang tersedia:

- **Microsoft Edge (Sangat Direkomendasikan untuk Windows):**
  flutter run -d edge
- **Google Chrome:**
  flutter run -d chrome
- **Mozilla Firefox / Safari / Browser Lain:**
  flutter run -d web-server
- **Smartphone / Emulator Android:**
  flutter run -d android
- **Smartphone / Simulator iOS (Khusus macOS):**
  flutter run -d ios

Pastikan aplikasi berhasil tampil di layar menampilkan AppBar *"Kesehatan Siaga Kota Bandung"* tanpa pesan error.

3. Fitur Hot Reload dan Hot Restart

Gunakan fitur Hot Reload (tekan tombol r pada terminal) untuk melihat pembaruan antarmuka secara instan setelah menyimpan perubahan berkas kode, atau gunakan Hot

Restart (tekan tombol R pada terminal) jika terjadi perubahan struktur state atau penambahan inisialisasi data.

4. Menjalankan Ulang Aplikasi Tanpa Koneksi Internet

Jika jendela browser atau emulator tidak sengaja tertutup, jangan menggunakan perintah biasa karena Flutter Web akan mencoba menghubungi server CDN online yang mengakibatkan aplikasi macet (blank white screen) jika tidak ada koneksi internet. Jalankan kembali aplikasi menggunakan perintah khusus mode offline berikut:

- Microsoft Edge (Windows):
  flutter run -d edge --no-web-resources-cdn --no-pub
- Google Chrome:
  flutter run -d chrome --no-web-resources-cdn --no-pub
- Mozilla Firefox / Safari / Browser Lain:
  flutter run -d web-server --no-web-resources-cdn --no-pub
- Android:
  flutter run -d android --no-pub
- iOS:
  flutter run -d ios --no-pub

D.2 Instruksi Umum Pengerjaan

1. Ketikkan kode pada masing-masing kasus di bawah ini ke dalam editor Flutter Anda.
2. Lengkapi bagian yang rumpang (_____) sesuai dengan petunjuk pada komentar bernomor.
3. Jalankan program dan pastikan tidak ada *error*.
4. Buat kesimpulan singkat disetiap akhir kasus.
5. Kumpulkan seluruh file hasil pengerjaan Anda di akhir sesi praktikum ke dalam LMS.

Contoh:

Aplikasi membutuhkan program yang dapat menyapa pengguna secara otomatis dan mencetak umur mereka. Anda diminta melengkapi tipe data dan perintah cetak layar.

```
void main() {
  // TODO 1: Lengkapi tipe data yang tepat untuk teks dan angka bulat
  ____ nama = "Budi";
  ____ umur = 20;

  // TODO 2: Lengkapi perintah bawaan Dart untuk mencetak ke layar
  ____("Halo, nama saya $nama dan umur saya $umur");

  // Kesimpulan pada soal ini adalah:
  // _____________________________________________________________
}
```

Jawaban:

```
void main() {
  // TODO 1: Lengkapi tipe data yang tepat untuk teks dan angka bulat
  String nama = "Budi";
  int umur = 20;

  // TODO 2: Lengkapi perintah bawaan Dart untuk mencetak ke layar
  print("Halo, nama saya $nama dan umur saya $umur");

  // Kesimpulan pada soal ini adalah:
  // String digunakan untuk teks, dan int digunakan untuk angka bulat.
}
```

D.3 Soal Praktikum

Dinas Kesehatan Kota Bandung bekerja sama dengan Bandung Command Center sedang mengembangkan aplikasi **Dashboard Layanan Kesehatan & Kedaruratan Siaga Kota Bandung**. Aplikasi ini bertujuan mempermudah warga dalam mencari fasilitas rujukan kesehatan, mengecek ketersediaan kamar, dan menghubungi layanan darurat secara cepat dan terintegrasi.

Tugas 1: Buatlah search bar dan daftar filter kategori fasilitas kesehatan dalam bentuk horizontal list.

```
Widget buildSearchAndCategoryBar() {
  final List<String> kategoriList = ['Semua', 'RSUD Siaga', 'Puskesmas', 'Klinik 24 Jam', 'Apotek'];

  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Container(
        padding: EdgeInsets.symmetric(horizontal: 14),
        decoration: BoxDecoration(
          color: Colors.grey.shade100,
          borderRadius: BorderRadius.circular(12),
          border: Border.all(color: Colors.grey.shade300),
        ),
        child: Row(
          children: [
            Icon(Icons.search, color: Colors.grey.shade600),
            SizedBox(width: 10),
            Expanded(
              child: TextField(
                decoration: InputDecoration(
                  hintText: 'Cari fasilitas kesehatan di Kota Bandung...',
                  border: InputBorder.none,
                ),
              ),
            ),
          ],
        ),
      ),
      SizedBox(height: 12),
      SizedBox(
        height: 38,
        child: ListView.builder(
          // TODO 1: Tentukan arah pengguliran secara horizontal
          scrollDirection: Axis._____,
          // TODO 2: Tentukan jumlah item sesuai panjang data kategoriList
          itemCount: kategoriList._____,
          // TODO 3: Tuliskan parameter index pada callback pembangun item
          itemBuilder: (BuildContext context, int _____) {
            // TODO 4: Cek apakah item pertama sedang terpilih (index bernilai 0)
            final bool isSelected = index == _____;

            return Container(
              margin: EdgeInsets.only(right: 8),
              padding: EdgeInsets.symmetric(horizontal: 16, vertical: 8),
              decoration: BoxDecoration(
                // TODO 5: Terapkan warna biru jika terpilih dan putih jika tidak terpilih
                color: isSelected ? Colors.blue : Colors._____,
                borderRadius: BorderRadius.circular(20),
                border: Border.all(
                  color: isSelected ? Colors.blue : Colors.grey.shade300,
                ),
              ),
```

```
            ),
            child: Center(
              child: Text(
                // TODO 6: Tampilkan teks kategori berdasarkan indeks saat ini
                kategoriList[_____],
                style: TextStyle(
                  fontSize: 12,
                  fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
                  // TODO 7: Terapkan warna teks putih jika terpilih dan hitam jika tidak terpilih
                  color: isSelected ? Colors.white : Colors._____,
                ),
              ),
            ),
          );
        },
      ),
    ),
  ],
);
}
```

Tugas 2: Buatlah menu tombol aksi cepat tanggap darurat (Ambulans 119, IGD 24 Jam, Donor Darah, Halo Dinkes) menggunakan grid 2 kolom.

```
Widget buildQuickActionGrid() {
  final List<Map<String, dynamic>> menuAksi = [
    {'title': 'Ambulans 119', 'icon': Icons.medical_services, 'color': Colors.red},
    {'title': 'IGD 24 Jam', 'icon': Icons.local_hospital, 'color': Colors.orange},
    {'title': 'Donor Darah PMI', 'icon': Icons.bloodtype, 'color': Colors.red.shade700},
    {'title': 'Halo Dinkes', 'icon': Icons.phone_in_talk, 'color': Colors.green},
  ];

  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        'Layanan Tanggap Darurat Kota Bandung',
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
      ),
      SizedBox(height: 10),
      GridView.builder(
        physics: NeverScrollableScrollPhysics(),
        // TODO 1: Aktifkan shrinkWrap agar ukuran tinggi menyesuaikan total isi anak
        shrinkWrap: _____,
        // TODO 2: Tentukan jumlah item dari panjang list menuAksi
        itemCount: menuAksi._____,
        gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
          // TODO 3: Tentukan jumlah kolom sebanyak 2
          crossAxisCount: _____,
          crossAxisSpacing: 10,
          mainAxisSpacing: 10,
          // TODO 4: Tentukan rasio aspek kartu sebesar 1.25
          childAspectRatio: _____,
        ),
        itemBuilder: (BuildContext context, int index) {
          // TODO 5: Ambil data map berdasarkan indeks perulangan
          final item = menuAksi[_____];

          return Card(
            elevation: 1.5,
            shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
            child: Padding(
              padding: EdgeInsets.all(12),
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
```

```
                CircleAvatar(
                  // TODO 6: Gunakan transparansi warna 0.12 dari item color
                  backgroundColor: item['color']._____(0.12),
                  radius: 22,
                  // TODO 7: Tampilkan icon dari key 'icon' pada map item
                  child: Icon(item['_____'], color: item['color'], size: 24),
                ),
                SizedBox(height: 8),
                // TODO 8: Tampilkan judul layanan dari key 'title' pada map item
                Text(
                  item['_____'],
                  style: TextStyle(fontWeight: FontWeight.bold, fontSize: 13),
                  textAlign: TextAlign.center,
                ),
              ],
            ),
          ),
        );
      },
    ),
  ],
);
}
```

Tugas 3: Buatlah direktori daftar fasilitas Kesehatan menggunakan list view vertikal.

```
Widget buildHospitalDirectory() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        'Fasilitas Kesehatan Rujukan Siaga',
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
      ),
      SizedBox(height: 10),
      ListView.builder(
        physics: NeverScrollableScrollPhysics(),
        // TODO 1: Aktifkan shrinkWrap agar batas tinggi terdefinisi di dalam Column
        shrinkWrap: _____,
        // TODO 2: Tentukan jumlah item sesuai panjang data mockHospitals
        itemCount: mockHospitals._____,
        itemBuilder: (BuildContext context, int index) {
          // TODO 3: Ambil objek model RumahSakit berdasarkan indeks perulangan
          final hospital = mockHospitals[_____];

          return Card(
            elevation: 2,
            margin: EdgeInsets.only(bottom: 10),
            shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
            child: ListTile(
              contentPadding: EdgeInsets.symmetric(horizontal: 14, vertical: 6),
              leading: CircleAvatar(
                backgroundColor: hospital.isIgdOpen ? Colors.blue.shade50 : Colors.red.shade50,
                // TODO 4: Tampilkan ikon rumah sakit (local_hospital)
                child: Icon(
                  Icons._____,
                  color: hospital.isIgdOpen ? Colors.blue : Colors.red,
                ),
              ),
              // TODO 5: Tampilkan nama rumah sakit dari properti objek hospital
              title: Text(
                hospital._____,
                style: TextStyle(fontWeight: FontWeight.bold, fontSize: 14),
              ),
```

```
            ),
            subtitle: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                SizedBox(height: 4),
                // TODO 6: Tampilkan alamat fasilitas kesehatan dari properti objek hospital
                Text(hospital._____, style: TextStyle(fontSize: 12)),
                SizedBox(height: 2),
                // TODO 7: Tampilkan sisa tempat tidur dari properti availableBeds
                Text(
                  'Sisa Tempat Tidur: ${hospital._____}',
                  style: TextStyle(fontSize: 11, color: Colors.grey.shade700),
                ),
              ],
            ),
            // TODO 8: Tampilkan teks badge status operasional (IGD BUKA jika true, IGD TUTUP jika false)
            trailing: Container(
              padding: EdgeInsets.symmetric(horizontal: 10, vertical: 5),
              decoration: BoxDecoration(
                color: hospital.isIgdOpen ? Colors.green.shade100 : Colors.red.shade100,
                borderRadius: BorderRadius.circular(8),
              ),
              child: Text(
                hospital.isIgdOpen ? '_____' : '_____',
                style: TextStyle(
                  fontSize: 10,
                  fontWeight: FontWeight.bold,
                  color: hospital.isIgdOpen ? Colors.green.shade800 : Colors.red.shade800,
                ),
              ),
            ),
          ),
        );
      },
    ),
  ],
);
}
```

Tugas 4: Buatlah daftar kontak nomor darurat yang dilengkapi garis pembatas (separated list) di bagian bawah halaman.

```
Widget buildEmergencySeparated() {
  final List<Map<String, String>> kontakDarurat = [
    {'nama': 'PSC 119 Kota Bandung', 'nomor': '119 / 022-119'},
    {'nama': 'Command Center Bandung', 'nomor': '112'},
    {'nama': 'Posko PMI Kota Bandung', 'nomor': '022-4207019'},
    {'nama': 'Dinas Kesehatan Siaga', 'nomor': '022-4203752'},
  ];

  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        'Nomor Penting Kedaruratan Kota Bandung',
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
      ),
      SizedBox(height: 10),
      Card(
        elevation: 2,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
        child: ListView.separated(
          physics: NeverScrollableScrollPhysics(),
          // TODO 1: Aktifkan shrinkWrap agar widget menyesuaikan ukuran konten
          shrinkWrap: _____,
          // TODO 2: Tentukan jumlah item dari panjang list kontakDarurat
          itemCount: kontakDarurat._____,
          // TODO 3: Kembalikan widget garis pemisah Divider
          separatorBuilder: (BuildContext context, int index) {
            return _____(height: 1, color: Colors.grey.shade300);
          },
```

```
            },
            itemBuilder: (BuildContext context, int index) {
              // TODO 4: Ambil map data kontak berdasarkan indeks perulangan
              final item = kontakDarurat[_____];

              return ListTile(
                // TODO 5: Tampilkan icon emergency dengan warna merah
                leading: Icon(Icons._____, color: Colors._____),
                // TODO 6: Tampilkan nama kontak darurat dari key 'nama'
                title: Text(item['_____']!, style: TextStyle(fontSize: 13, fontWeight: FontWeight.bold)),
                // TODO 7: Tampilkan nomor telepon darurat dari key 'nomor'
                subtitle: Text(item['_____']!, style: TextStyle(fontSize: 12, color: Colors.blue.shade800)),
                trailing: IconButton(
                  // TODO 8: Tampilkan icon telepon dengan warna hijau
                  icon: Icon(Icons._____, color: Colors._____),
                  onPressed: () {
                    print('Menghubungi ${item['nama']}');
                  },
                ),
              );
            },
          ),
        ),
      ],
    );
  }
```

E. Tes Akhir

Dinas Perdagangan dan Perindustrian bersama Dinas Koperasi dan UMKM Kota Bandung sedang merancang prototipe antarmuka mobile Pemberdayaan Sentra Industri Rakyat & UMKM Kota Bandung. Aplikasi ini bertujuan memperkenalkan dan memantau sentra-sentra industri rakyat unggulan Kota Bandung (Sentra Sepatu Cibaduyut, Sentra Rajut Binong Jati, Sentra Kaos Suci, Sentra Kuliner Pasirkoja, dll.).

Spesifikasi Fungsional & Kebutuhan Antarmuka:

1. **Identitas Resmi & Header Aplikasi:**
   - Gunakan AppBar dengan judul *"Sentra UMKM & Kreatif Bandung"*,
   - Buat banner ringkasan di bawah AppBar yang menampilkan komitmen pemberdayaan sentra industri rakyat dan informasi jumlah perajin aktif binaan Kota Bandung.



2. **Klaster Unggulan Sentra Industri:**
   - Bangun menu ringkasan 4 klaster utama menggunakan **GridView.builder** (2 kolom) dengan rasio aspek proporsional (childAspectRatio: 2.2 hingga 2.5).
   - 4 Klaster yang wajib ditampilkan:
     - *Kerajinan Sepatu & Kulit* (Kawasan Cibaduyut)
     - *Industri Tekstil & Rajut* (Kawasan Binong Jati)
     - *Sablon & Kaos Kreatif* (Kawasan Suci / Surapati)

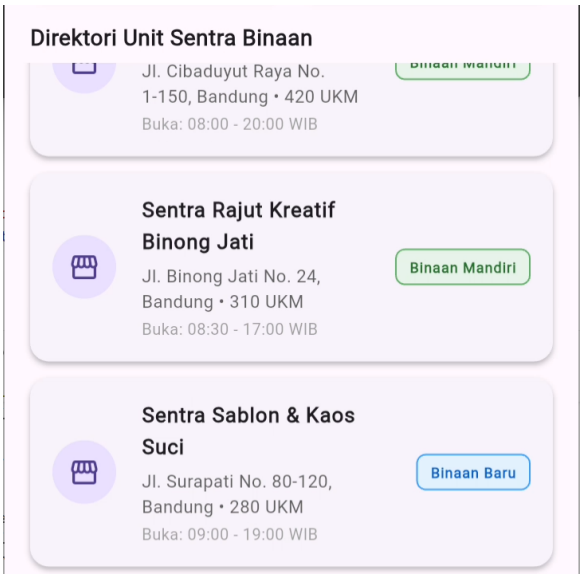
- *Kuliner Olahan Rakyat* (Kawasan Pasirkoja)

- Setiap kartu grid memuat ikon kategori yang representatif, nama klaster, dan jumlah unit usaha UKM aktif.



3. Direktori Unit Sentra Binaan:

- Siapkan kumpulan data daftar sentra industri menggunakan struktur data koleksi List<Map<String, dynamic>>
  (memuat *key*: nama, kawasan, jumlahPerajin, statusBinaan, dan jamBuka).
- Tampilkan minimal 3–5 data sentra industri nyata Kota Bandung menggunakan **ListView.builder**.
- Setiap elemen daftar dibungkus dalam Card dengan penataan ListTile rapi:
  - leading: Avatar melingkar berikon representatif industri.
  - title: Nama sentra industri (*bold*).
  - subtitle: Kawasan jalan di Kota Bandung dan jumlah unit perajin aktif.
  - trailing: *Badge* status binaan (*"Binaan Mandiri"* berlatar hijau, *"Binaan Baru"* berlatar biru).



F. Jurnal Praktikum

Untuk pengerjaan **Tugas Praktikum dan Tes Akhir** di atas, Anda diwajibkan menyusun Jurnal Praktikum dalam format dokumen (pdf atau word) dengan struktur:

1. Halaman Sampul: Nama, NIM, Kelas, dan Judul Praktikum.
2. Screenshot hasil dari **Tugas Praktikum dan Tes Akhir.**

3. Pembahasan dan Analisis **Program Tugas Akhir**:
   Jelaskan alur kerja program yang Anda buat. Anda dapat menyertakan potongan kode (snippet) pada bagian-bagian penting. Lampirkan screenshot dari output dan berikan analisis singkat mengenai hasil output tersebut
4. Kesimpulan Praktikum
   Tuliskan kesimpulan menyeluruh mengenai apa yang telah Anda pelajari pada sesi praktikum hari ini.

G. Cara Pengumpulan

1. Buatlah sebuah folder baru di komputer Anda dan beri nama dengan format: pekan8_NIM_NamaLengkap (Contoh: pekan8_120220001_BudiSantoso).
2. Masukkan proyek flutter dan seluruh file program Praktikum serta Tugas Akhir ke dalam folder tersebut.
3. Sebelum memasukkan proyek flutter ke folder, jalankan perintah **flutter clean** pada terminal.
4. Masukkan juga file Jurnal Praktikum ke folder tersebut.
5. Compress atau arsipkan folder yang sudah berisi file-file tersebut menjadi satu file berformat .zip atau .rar.
6. Upload ke LMS.